

AULA 000

Introdução aos Paradigmas de Programação

Resumo direto para leitura rápida

IDEIA CENTRAL

Paradigmas são diferentes formas de pensar, organizar e expressar soluções computacionais. Conhecê-los ajuda a escolher ferramentas melhores, aprender novas linguagens mais rápido e escrever programas com mais clareza.

1. Por que estudar paradigmas?

- **Ampliar a capacidade de resolver problemas:** Cada paradigma oferece uma forma diferente de modelar uma solução.
- **Escolher melhor a linguagem:** A linguagem adequada depende do problema, da equipe, do desempenho e do ecossistema.
- **Aprender novas linguagens com mais facilidade:** Quando os conceitos são conhecidos, a sintaxe deixa de ser o principal obstáculo.
- **Evitar o uso de uma única ferramenta para tudo:** Uma linguagem pode funcionar em vários contextos, mas nem sempre será a opção mais clara, segura ou produtiva.

Por que existem tantas linguagens?

Porque os problemas são diferentes. Computação científica, sistemas comerciais, inteligência artificial, sistemas operacionais, aplicações web, jogos e sistemas embarcados possuem necessidades distintas. Além disso, novas linguagens surgem com a evolução do hardware, das práticas de engenharia e das demandas de segurança e produtividade.

2. Evolução das linguagens de programação

A evolução não ocorreu por substituição completa, mas por acumulação de ideias. Linguagens novas reaproveitam conceitos anteriores e introduzem outras formas de organizar programas.

Linguagem / família	Contribuição principal	Contexto típico
Fortran	Popularizou a programação de alto nível	Computação científica e numérica
LISP	Fortaleceu ideias funcionais e simbólicas	Inteligência artificial e pesquisa
ALGOL	Influenciou a estrutura e a sintaxe de muitas linguagens	Algoritmos e ensino
COBOL	Foco em processamento de dados comerciais	Sistemas empresariais
C e derivados	Controle de memória, desempenho e sistemas	Sistemas operacionais e aplicações
Smalltalk / Java / C#	Consolidação da orientação a objetos	Aplicações corporativas e interfaces
Prolog	Programação baseada em fatos e regras	Raciocínio lógico e IA simbólica

3. Sintaxe, semântica e tradução do programa

Sintaxe

É a forma correta de escrever construções na linguagem. Inclui palavras reservadas, símbolos, parênteses, separadores e regras de organização. Um erro sintático impede que o compilador ou interpretador reconheça o programa.

Semântica

É o significado do que foi escrito. Um programa pode estar sintaticamente correto e, ainda assim, produzir um resultado errado. A semântica dinâmica descreve o comportamento das instruções durante a execução.

Etapas básicas de análise

1. **Análise léxica:** Transforma o texto em tokens, como nomes, números, operadores e palavras reservadas.
2. **Análise sintática:** Verifica como os tokens se combinam e constrói uma representação estrutural, geralmente uma árvore sintática.
3. **Análise semântica:** Verifica tipos, declarações, usos válidos de nomes e outras regras de significado.
4. **Execução ou geração de código:** O programa é interpretado ou convertido para uma forma executável.

4. Nomes, tipos, escopo e tempo de vida

Conceito	Definição direta
Nome	Identificador usado para referenciar uma variável, função, classe, módulo ou outro elemento.
Vinculação	Associação entre um nome e uma entidade, valor, tipo ou posição de memória.
Escopo	Região do programa em que um nome pode ser acessado.
Tempo de vida	Período durante o qual a entidade existe na memória durante a execução.
Tipo de dado	Conjunto de valores e operações permitidas sobre esses valores.

Por que os tipos importam?

O sistema de tipos reduz ambiguidades e previne operações inválidas. Ele define quais valores podem ser armazenados e quais operações fazem sentido, como somar números, comparar textos ou acessar elementos de uma coleção.

5. Expressões e estruturas de controle

Avaliação em curto-circuito

Em uma expressão lógica, a segunda parte pode não ser executada quando a primeira já determina o resultado. Exemplo: em `x != 0 && 10 / x > 2`, se `x` for zero, a primeira condição será falsa e a divisão não será realizada, evitando erro.

Estruturas de controle

- Seleção: `if`, `else`, `switch` e construções equivalentes.
- Repetição: `for`, `while`, `do-while` e iterações sobre coleções.
- Subprogramas: funções, procedimentos, parâmetros e retorno.
- Recursão: uma função chama a si mesma para resolver versões menores do problema.
- Controle excepcional: tratamento de erros e exceções.

6. Principais paradigmas

Paradigma	Ideia principal	Como pensa a solução	Exemplos
Imperativo	Sequência de comandos que altera o estado	"Faça isto, depois aquilo"	C, Pascal, Fortran

Paradigma	Ideia principal	Como pensa a solução	Exemplos
Orientado a objetos	Objetos combinam dados e comportamentos	Modela entidades, responsabilidades e colaboração	Java, C#, Smalltalk
Funcional	Funções, expressões e redução de efeitos colaterais	Transforma valores sem depender tanto de estado mutável	Haskell, Lisp, F#
Lógico	Fatos, regras e consultas	Declara o que é verdadeiro e pede ao sistema para deduzir respostas	Prolog
Concorrente	Várias tarefas avançam simultaneamente ou de forma intercalada	Coordena processos, threads, mensagens e sincronização	Go, Erlang, Rust

Linguagens multiparadigma

Uma linguagem moderna pode combinar vários paradigmas. Python e JavaScript misturam recursos imperativos, orientados a objetos e funcionais. Rust combina programação imperativa e funcional com forte preocupação com segurança de memória. O importante é escolher a abordagem adequada para cada parte do problema.

7. Programação com inteligência artificial

Ferramentas de IA podem gerar código, testes e documentação a partir de instruções em linguagem natural. Isso não elimina a necessidade de compreender programação. O desenvolvedor continua responsável por definir o problema, revisar decisões, testar o comportamento, avaliar segurança, verificar desempenho e manter o sistema.

Ao usar IA para programar, verifique:

- correção da lógica
- segurança e tratamento de dados
- testes e casos extremos
- desempenho e legibilidade
- adequação ao contexto do projeto

8. Síntese final

- Não existe uma linguagem perfeita para todos os problemas.
- Paradigmas são formas de pensar e estruturar soluções.
- Sintaxe define como escrever; semântica define o significado.
- Escopo, tipos, tempo de vida e controle de fluxo afetam diretamente a qualidade do programa.
- Linguagens modernas normalmente combinam vários paradigmas.
- A IA auxilia a produção de código, mas não substitui a compreensão e a responsabilidade técnica.

PENSAR DIFERENTE • ESCOLHER MELHOR • PROGRAMAR COM PROPÓSITO